



AESOP XI: Sovereign Edge-Computing Architecture (Final Master Specification)



6. Part 4: Multi-Node Compute Topology & Asymmetric Auditing Mesh

The physical compute cluster distributes memory, inference, and verification across distinct hardware nodes linked over a peer-to-peer Tailscale mesh network.





The Multi-Agent Home Node Swarm (Node Beta Dedicated Daemons)

To eliminate single-agent bottlenecks and cognitive decay, Node Beta hosts a partitioned suite of background agents, each bound to an isolated task profile:

1. **Home Assistant (Cross-Agent Auditor & Housekeeper):**
 - **Live Multi-Model Auditing:** Monitors live inference streams between the 0.8B and 9B models, logging every prompt, intermediate tool call, and raw execution output in real time.
 - **Daily Log Deduplication & Script Compilation:** Consolidates daily telemetry traces, strips redundant token overhead, cleans execution artifacts, and drafts automated system scripts from successful multi-step tool calls.
 - **Dataset Accumulation:** Packages continuous task-output-verdict triples into local .jsonl staging buffers for subsequent training validation.
2. **Red Agent Auditor (Air-Gapped Batch Training Guardrail):**
 - **Threshold-Triggered Intercept:** Does **not** intercept live conversational latency; it executes *only* when staged execution traces reach the defined context training threshold.
 - **Nope Data Bank Verification:** Compares the batch dataset against the nope_data_bank.json repository to quarantine toxic loops, hallucinated tool calls, corrupted extractors, and sensitive records.
 - **Cryptographic Sign-Off:** Once verified, the Red Agent signs the dataset and releases it for cloud ingestion.
3. **IT Help Desk & Manual Operator Agent:**
 - Maintains an active index of all hardware manuals (Qualcomm QAIRT, Android Media SDK, Linux kernel configs) and tool schemas.
 - Acts as an automated diagnostic assistant to identify runtime errors, script failures, and hardware throttle events.
4. **Web Ingestion & News Monitor Agent:**
 - Continuously tracks upstream releases, trending Hugging Face open-weight model architectures (GGUF quantizations), and open-source tool libraries.
5. **NPU / Local Inference Manager:**
 - Monitors temperature, dynamic RAM bounds (8.0–11.5 GB limit on Node Alpha), and NPU tensor offloading efficiency across the cluster.

7. Part 5: GCP Asymmetric Cross-Account IAM Handshake Protocol

To protect intellectual property behind business workspace firewalls while utilizing personal developer credit pools (\$1,000 Vertex AI Agent Builder credits), the system implements an asymmetric cross-account peering architecture.

The diagram illustrates two main resource tiers:

- BUSINESS / CONTRACTOR RESOURCE TIER**:
 - Project: Business Secure Workspace
 - Resource: gs://business-vault-bucket
 - Houses: Raw IP, Source Code, Logs
- PERSONAL DEVELOPER CREDIT ACCOUNT**:
 - Project: Personal Credit Account
 - Holds: \$1,000 Vertex AI Credit Pool
 - Hoss: Discovery Engine / Cloud Run

A central component labeled **[CROSS-ACCOUNT IAM BRIDGE]** connects the two tiers.

Below the bridge, three service account roles are listed:

- Service Account: service-PROJECT_NUMBER@gcp-sa-discoveryengine..
- Permission Role: roles/storage.objectViewer (READ-ONLY)
- Execution: Zero-Copy Indexing (Absorbed by Credits)

Core Architectural Rules

1. **Zero-Copy Ingestion:** Raw documentation and proprietary logs remain strictly inside `gs://business-secure-vault-bucket/`. No data files are duplicated or migrated to the personal developer tier.
2. **Compute-Credit Redirection:** The personal project initiates all Discovery Engine indexing and Grounded Generation API calls, ensuring all billing SKU charges hit the \$1,000 developer credit pool.
3. **Instant Access Severance:** Access can be revoked instantly by removing the personal Service Account from the business bucket's IAM Access Control List (ACL).

Production Handoff Bootstrap Script (gcp_cross_account_handshake.sh)

```
Bash
#!/usr/bin/env bash
#
# AESOP XI: CROSS-ACCOUNT RESOURCE INGESTION & CREDIT HARNESS
# Subsystem: Cloud-to-Edge Data Ingestion & Credit-Funded RAG Auditing
```

```

#
set -euo pipefail

RED='\033[0;31m'
GREEN='\033[0;32m'
CYAN='\033[0;36m'
YELLOW='\033[1;33m'
NC='\033[0m'

clear

echo -e "${CYAN}===== ${NC}"
echo -e "${CYAN} AESOP XI: GCP CROSS-ACCOUNT INGESTION & IAM PERMISSION BRIDGE ${NC}"
echo -e "${CYAN}===== ${NC}"

if command -v gcloud >/dev/null then
    echo -e "${RED}X ERROR: Google Cloud SDK (gcloud) is not installed in this shell.${NC}"
    exit
fi

echo -e "\n${YELLOW}[STEP 1/3] Fetching Personal Credit-Consumer Project Details...${NC}"
read -p "Enter your PERSONAL GCP Project ID (holding credits): " PERSONAL_PROJECT_ID

gcloud config set project "$PERSONAL_PROJECT_ID" &> /dev/null

PROJECT_NUMBER=$(gcloud projects list --filter="projectId=$PERSONAL_PROJECT_ID"
--format="value(projectNumber)")
if [ -z "$PROJECT_NUMBER" ] then
    echo -e "${RED}X ERROR: Failed to retrieve project number for $PERSONAL_PROJECT_ID.${NC}"
    exit
fi

SERVICE_ACCOUNT="service-${PROJECT_NUMBER}@gcp-sa-discoveryengine.iam.gserviceaccount.com"
echo -e "${GREEN}✓ Personal Project Number: ${PROJECT_NUMBER}${NC}"
echo -e "${GREEN}✓ Personal Vertex AI Service Account: ${SERVICE_ACCOUNT}${NC}"

echo -e "\n${YELLOW}[STEP 2/3] Configuring Read-Only Access on Business Resource
Bucket...${NC}"
read -p "Enter the secure GCS Bucket Name (e.g., business-secure-vault-bucket): "

```

```

BUCKET_NAME
BUCKET_NAME ${BUCKET_NAME#gs://}

echo -e "\nSelect Execution Mode:"
echo -e "1) ${CYAN}Execute Now:${NC} Apply IAM update directly (Requires Business Admin auth)."
echo -e "2) ${CYAN}Generate Handoff Block:${NC} Output CLI command for Business
Administrator:"
read -e "Select [1 or 2]: " EXEC_MODE

if "$EXEC_MODE" == "1" then
    echo -e "\nApplying policy binding on GCS bucket: ${YELLOW}gs://${BUCKET_NAME}${NC}..."
    gcloud storage buckets add-iam-policy-binding "gs://${BUCKET_NAME}"
        --member="serviceAccount:${SERVICE_ACCOUNT}"
        --role="roles/storage.objectViewer"
    echo -e "${GREEN}✓ SUCCESS: Cross-account permission bridge active!${NC}"
else
    echo -e "\n${CYAN}===== BUSINESS ADMIN HANDOFF COMMAND
===== ${NC}"
    echo -e "${YELLOW}gcloud storage buckets add-iam-policy-binding gs://${BUCKET_NAME} \"
    echo -e "  --member=\"serviceAccount:${SERVICE_ACCOUNT}\" \" \"
    echo -e "  --role=\"roles/storage.objectViewer\" \"${NC}"
    echo -e
    "${CYAN}===== ${NC}
C)"
fi

echo -e "\n${YELLOW}[STEP 3/3] Finalizing Data Store Creation Instructions...${NC}"
echo -e "1. Open the Personal Vertex AI Agent Builder Console."
echo -e "2. Create a new Data Store pointing to: ${GREEN}gs://${BUCKET_NAME}/${NC}"
echo -e "3. Select 'Unstructured Documents' and enable 'Continuous Import'."
echo -e
"${CYAN}===== ${NC}
C)\n"

```

8. Part 6: Continuous Improvement RLVR Training Flywheel

By decoupling inference from heavy compute, the home node remains lightweight while GCP instances handle continuous fine-tuning loops.

[REDACTED]

1. EDGE EXECUTION: Task Ingress on Node Alpha / Beta

[REDACTED]

[REDACTED]

2. LIVE AUDIT: Home Assistant compiles task triples

[REDACTED]

[REDACTED]

3. GATEWAY CHECK: Red Agent validates vs Nope DB

[REDACTED]

[REDACTED]

4. CLOUD SYNC: Verified .jsonl batches export to GCS

[REDACTED]

[REDACTED]

5. TRAINING RUN: GCP Vertex/Compute runs GRPO (Unsloth)

[REDACTED]



Operational Protocols

- **Base Weight Stability:** The base model architecture is permanently standardized on Qwen 3.5 (0.8B for execution, 9B for reasoning/querying) to prevent fine-tuning resets.
- **Reinforcement Learning via Verifiable Rewards (RLVR):** Uses GRPO (Group Relative Policy Optimization) executed via Unsloth/Axolotl on GCP A100/H100 instances. Training runs evaluate deterministic outcomes (code syntax accuracy, tool execution success, JSON schema adherence) rather than subjective prose.
- **Scheduled Synchronization:** Fine-tuning jobs trigger automatically upon hitting a 5,000 verified task threshold or run on a weekly maintenance schedule, compiling new GGUF Q4_0 quantizations for Node Alpha.



9. Part 7: Canonical Verification Schemas & File

Audit Utilities

I. Universal Capability Verification Schema
(skill_onboarding_schema.json)

JSON

```

|
|
"$schema" "https://json-schema.org/draft/2020-12/schema"
"title" "AESOP_XI_Universal_Skill_Onboarding_Schema"
"type" "object"
"properties"
  "skill_identity"
    "type" "object"
    "properties"
      "technical_identifier" "type" "string"
      "harness_origin"
        "type" "string"
        "enum" "POCOCK_SKILLS" "PRIME_AGENT_RLM" "ECC_FRAMEWORK" "GSD_CORE"
        "HONEY_DEV" "CUSTOM_LOCAL"
      "documentation_reference_path" "type" "string"
    "required" "technical_identifier" "harness_origin" "documentation_reference_path"
  "runtime_execution_target"
    "type" "object"
    "properties"
      "designated_node" "type" "string" "enum" "NODE_ALPHA_PHONE" "NODE_BETA_JETSON"
      "NODE_GAMMA_PI" "NODE_DELTA_GCP"
      "execution_subsystem" "type" "string" "enum" "HORIZONS_UI_NANOMODEL"
      "SHELL_DAEMON_OS" "MEDIA_DAEMON_VISION" "JETSON_CUDA_RUNTIME"
      "memory_route" "type" "string" "enum" "MEMO_EPISODIC" "OB1_POSTGRES_VECTORS"
      "LOCAL_OBSIDIAN_VAULT" "NOTEBOOKLM_PY"
    "required" "designated_node" "execution_subsystem" "memory_route"
  "permission_and_safety_bounds"
    "type" "object"
    "properties"
      "requires_shell_access" "type" "boolean"
      "requires_screen_capture" "type" "boolean"
      "nope_bank_exclusion_check" "type" "boolean"
    "required" "requires_shell_access" "requires_screen_capture" "nope_bank_exclusion_check"
  "required" "skill_identity" "runtime_execution_target" "permission_and_safety_bounds"
|

```


II. Manual Audit JSON-RPC Telemetry Trace Schema (trace_exchange_schema.json)

JSON

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "AESOP_XI_Manual_Audit_Trace_Schema",
  "type": "object",
  "properties": {
    "jsonrpc": {
      "type": "string",
      "enum": "2.0"
    },
    "method": {
      "type": "string",
      "enum": "audit.execution_trace.submit"
    },
    "params": {
      "type": "object",
      "properties": {
        "originating_node_id": {
          "type": "string",
          "enum": "NODE_ALPHA_RAZR"
        },
        "NODE_BETA_JETSON": "NODE_GAMMA_PI",
        "target_repository_layer": {
          "type": "string",
          "enum": "horizons-ui-v2.0" "nova-daemon-shell" "nova-daemon-media"
        },
        "file-management-and-skills": "obsidian-knowledge-vault" "agent-harness-hub"
      }
    },
    "execution_metadata": {
      "type": "object",
      "properties": {
        "timestamp_epoch": {
          "type": "integer"
        },
        "tool_invoked": {
          "type": "string"
        },
        "raw_command": {
          "type": "string"
        }
      },
      "required": "timestamp_epoch" "tool_invoked" "raw_command"
    },
    "inference_payload": {
      "type": "object",
      "properties": {
        "system_prompt_version": {
          "type": "string"
        },
        "reasoning_steps": {
          "type": "string"
        },
        "raw_output": {
          "type": "string"
        }
      },
      "required": "system_prompt_version" "reasoning_steps" "raw_output"
    }
  }
}
```

```

    ]
    "system_telemetry"
    "type" "object"
    "properties"
        "stdout_logs" "type" "string"
        "stderr_logs" "type" "string"
        "exit_code" "type" "integer"
    ]
    "required" "stdout_logs" "stderr_logs" "exit_code"
]
]
    "required" "originating_node_id" "target_repository_layer" "execution_metadata"
"inference_payload" "system_telemetry"
]
    "id" "type" "integer"
]
    "required" "jsonrpc" "method" "params" "id"
]

```

III. Local Repository Verifier Utility (audit_layer_bounds.py)

```

Python
#!/usr/bin/env python3
"""
AESOP XI PHYSICAL REPOSITORY ARCHITECT & REPO VERIFIER
Audits directory boundaries, verifies agent manifests, and enforces strict repository isolation.
"""

import os
import sys
import json

TARGET_DIRECTORIES = [
    "horizons-ui-v2.0"
    "nova-daemon-shell"
    "nova-daemon-media"
    "file-management-and-skills"
    "agent-harness-hub"
    "localai-omni-route"
]

```

```
"obsidian-knowledge-vault"  
"red-agent-auditor"  
"node-beta-jetson"  
"gcp-training-flywheel"  
|
```

```
def enforce_ground_truth_layout():  
    print("⌚ [SYSTEM AUDIT] Verifying AESOP XI directory trees and manifest profiles...")  
    anomalies_patched = 0  
  
    for folder in TARGET_DIRECTORIES:  
        if not os.path.isdir(folder):  
            print(f"⚠️ Directory missing: Creating layer '{folder}'...")  
            os.makedirs(folder, exist_ok=True)  
            anomalies_patched += 1  
  
        manifest_path = os.path.join(folder, "agent_manifest.json")  
        if not os.path.exists(manifest_path):  
            print(f"⚠️ Manifest missing in '{folder}': Generating isolation blueprint...")  
            manifest_payload = {  
                "repository_scope_id": folderupper(),  
                "assigned_building_agent": f"{folder}-specialist",  
                "cross_contamination_block": True,  
                "output_commit_restriction": "HUMAN_AIR_GAP_ONLY",  
                "jsonl_marker_index_active": True,  
            }  
            with open(manifest_path, "w", encoding="utf-8") as f:  
                json.dump(manifest_payload, f, indent=4)  
            anomalies_patched += 1  
  
    print(f"✅ AUDIT COMPLETE: {anomalies_patched} structural anomalies patched. Repositories  
locked to blueprint.")  
  
if __name__ == "__main__":  
    enforce_ground_truth_layout()
```